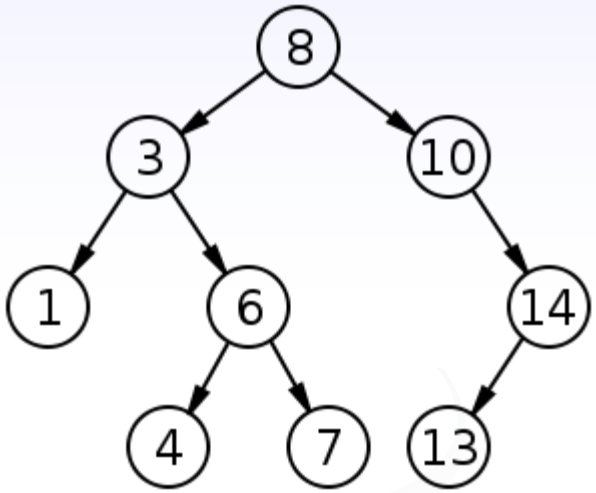
Двоичное дерево

Двоичное дерево – это *нелинейный двухсвязный список* представляющий конечное множество элементов, каждый из которых может быть либо пустым, либо содержать корневой узел и, возможно, другие узлы (генеалогическое дерево, дерево поиска, дерево арифметического выражения).

Рекурсивная структура данных



Двоичное дерево

Каждый узел двоичного дерева может иметь 0, 1 или 2 поддерева (потомка).

Если у узла нет поддеревьев (потомков), то он называется листом.

Каждый узел двоичного дерева имеет идентифицирующий его ключ.

Двоичное дерево поиска

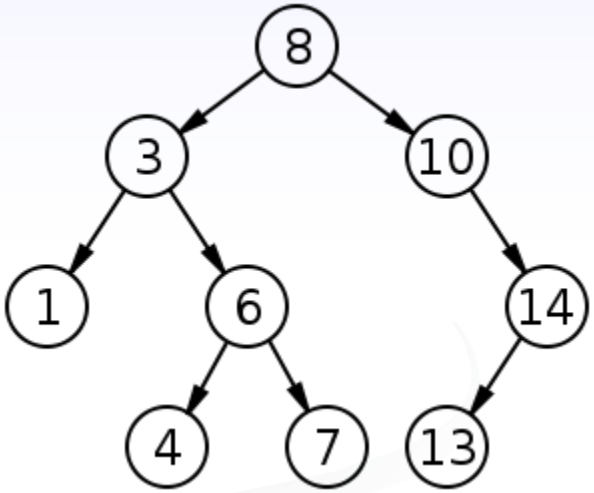
Дерево поиска является разновидностью двоичных деревьев.

Свойство:

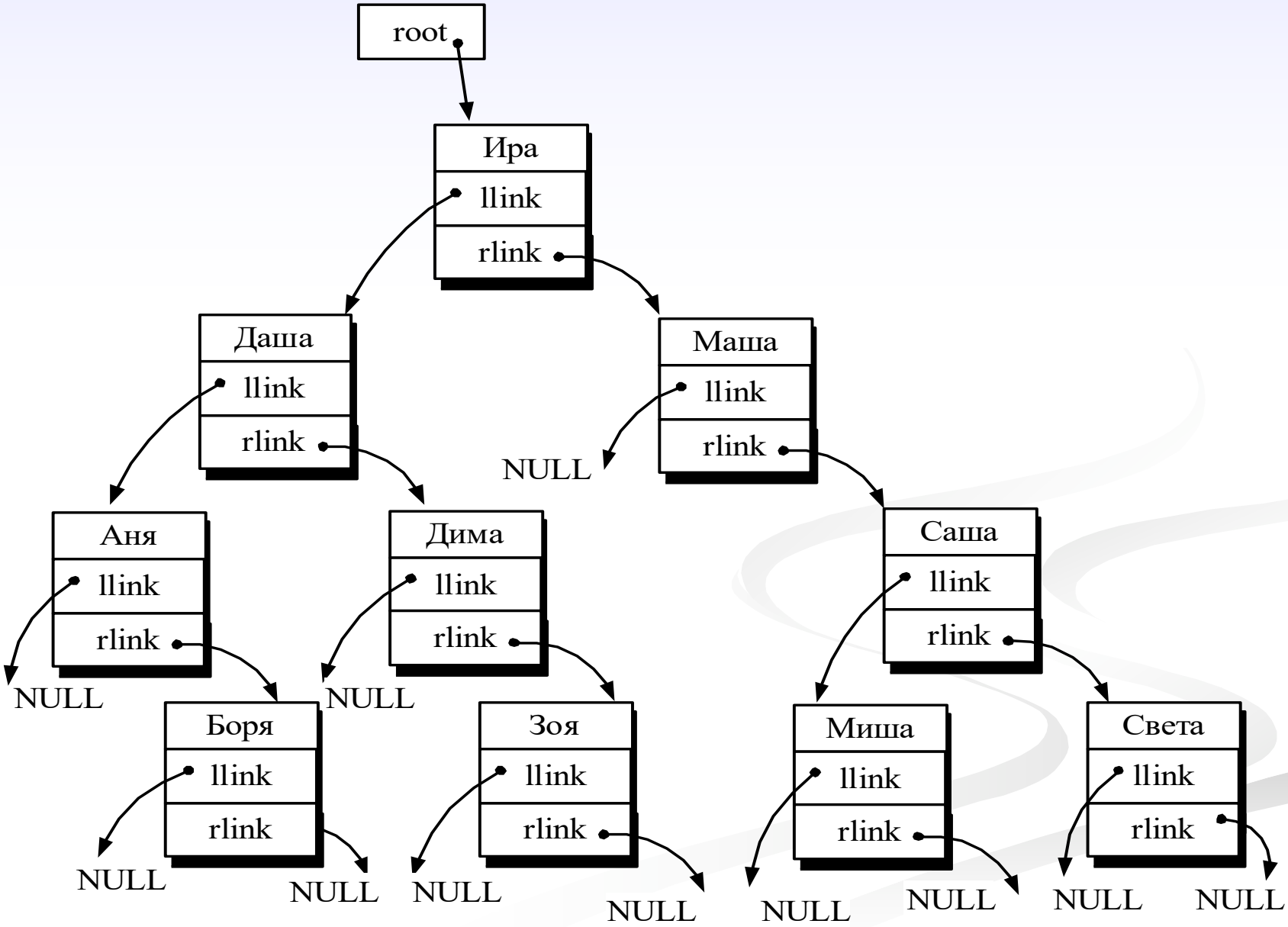
все узлы дерева, лежащие *левее и ниже* данного узла, имеют значение ключа *меньшее*, чем значение ключа данного узла,

а все узлы, лежащие *правее и ниже* данного узла, имеют значение ключа *большее*, чем значение ключа данного узла.

Приведенное свойство позволяет *эффективно организовывать поиск O(logN) и сортировку O(N) по двоичному дереву*.



Двоичное дерево поиска



Двоичное дерево поиска

Сложность **алгоритмов, используемых для добавления, исключения и поиска элементов в дереве, зависит не от количества узлов, а от *количества уровней* дерева.**

Например, относительно сбалансированное дерево с *1 000 000* узлов имеет лишь *20* уровней.

Двоичное дерево поиска

Следующая программа демонстрирует **построение и обход** двоичного дерева.

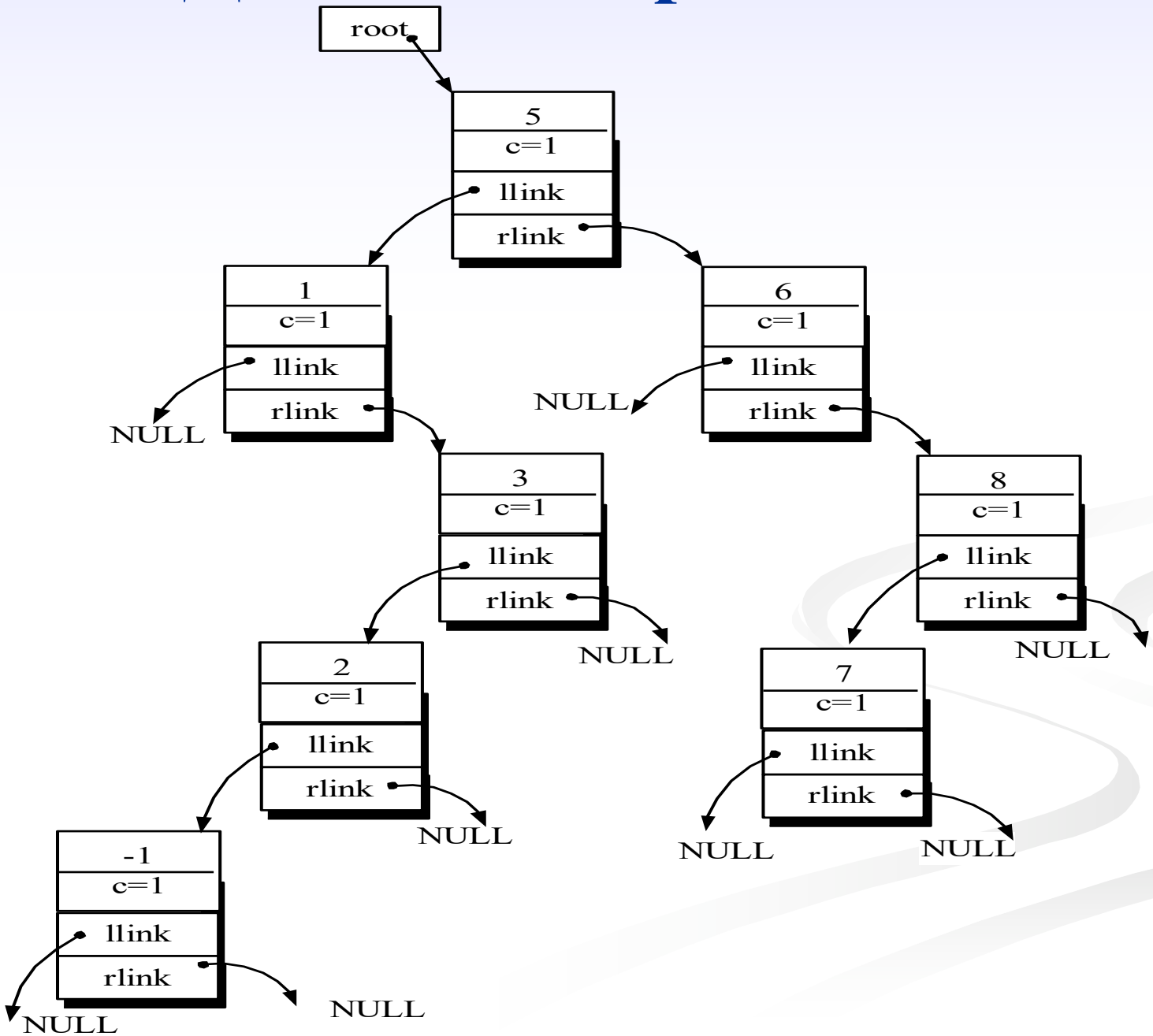
Дерево строится из входной последовательности чисел, и эти числа используются как ключ при построении. Следует заметить, что каждый узел дерева содержит четыре поля:

info - число из входной последовательности,

c - счетчик числа вхождений данного ключа в исходную последовательность чисел,

rlink и llink - указатели на левое и правое поддерево соответственно.

Двоичное дерево поиска



Двоичное дерево поиска

Основные операции:

1. **Построение**
2. **Добавление**
3. **Удаление**
4. **Поиск**
5. **Сортировка**
6. **Балансировка**

Двоичное дерево поиска

```
struct node // Структура узла
      {
       int info ;
       int c ;
       node *left,
            *right;
      };
```

Двоичное дерево поиска

```
// -------Функция построения дерева---------
 node *tree(node *p, int w)
  {
    if (p == NULL)
     { p = new node;
       p->info = w;
       p->left = NULL;
       p->right = NULL;
       p->c = 1;
     }
    else
      if (w == p->info)
       p->c = p->c + 1;
      else
      if (w < p->info)
       p->left = tree (p->left, w);
       else p->right = tree (p->right, w);
   return p;
 }
```

Двоичное дерево поиска

//---------Функция обхода дерева ----------

```
void treeprint (node *p)
{
if (p!=NULL)
{
  treeprint(p->left);
  printf("%d\t%d\n", p->c, p->info);
 treeprint(p->right);
}
}
```

Двоичное дерево поиска

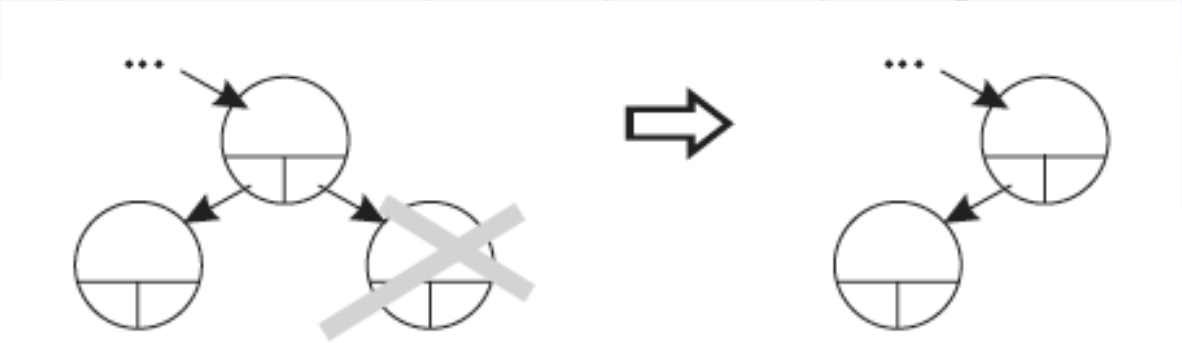
```
node * tree_find (node *p, int x)
 {
     while ( p != NULL )
     {if ( x == p->info )
         return p;
      if ( r>p->info ) p=p->right;
        else p = p->left;
     }
      return NULL;
  }
```

Двоичное дерево поиска

```
void main ( )
{     node *root;
      int w;
      root = NULL;
      scanf ("%d", &w);
      while ( !feof ( stdin ))
       {    root = tree (root,w);
           scanf("%d", &w);
       }
      treeprint(root);
if ( tree_find ( root, 5 ) )
           printf ( "\n Yes" );
     else printf ( "\n No" );
}
```

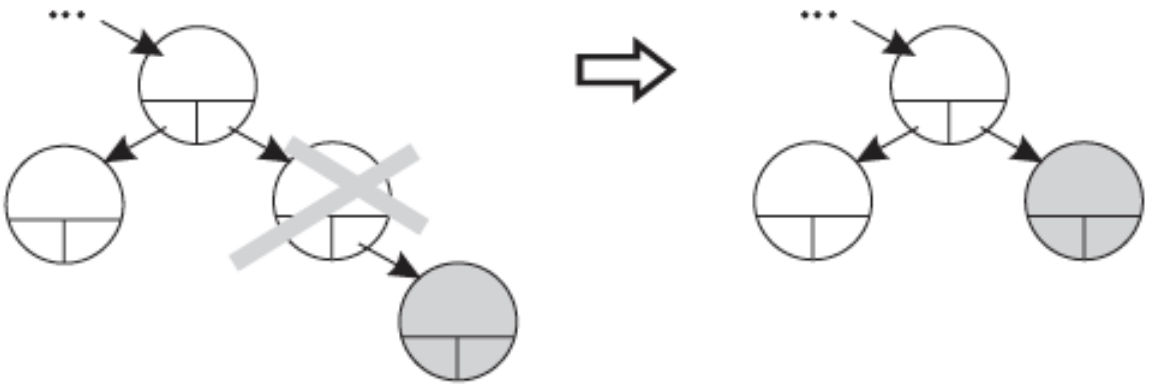
Удаление узла из двоичного дерева поиска

- Если узел является листом, то есть не имеет потомков, достаточно обнулить соответствующий указатель узла-предка



а)

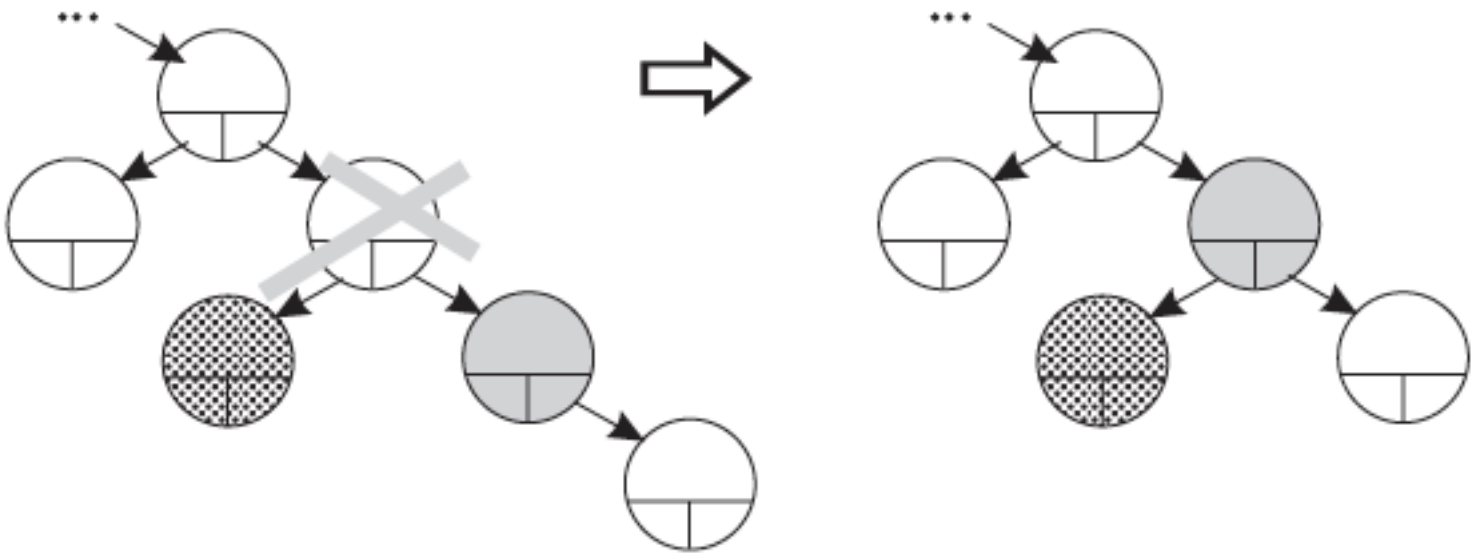
- Если узел имеет только одного потомка, то этот потомок ставится на место удаляемого узла, а в остальном дерево не изменяется



б)

Удаление узла из двоичного дерева поиска

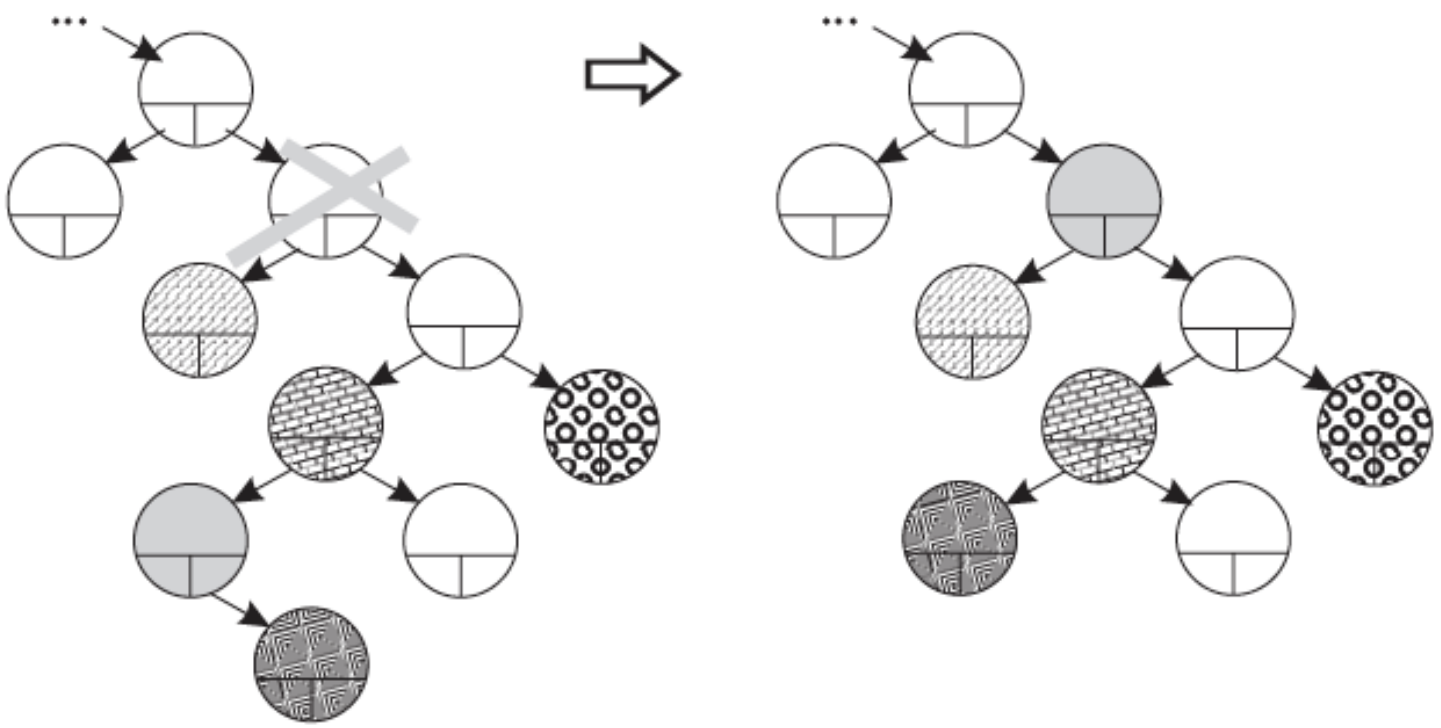
- Хуже всего, когда у узла есть оба потомка, но и здесь есть простой особый случай: если у его правого потомка нет левого потомка, удаляемый узел заменяется своим правым потомком, а левый потомок удаляемого узла подключается вместо отсутствующего левого потомка.



в)

Удаление узла из двоичного дерева поиска

- В общем же случае на место удаляемого узла помещается самый левый лист его правого поддерева (или наоборот — самый правый лист его левого поддерева).
- Корень дерева удаляется аналогичным образом за исключением того, что заменяющий его узел не требуется подсоединять к узлу-предку. Вместо этого обновляется указатель на корень дерева.



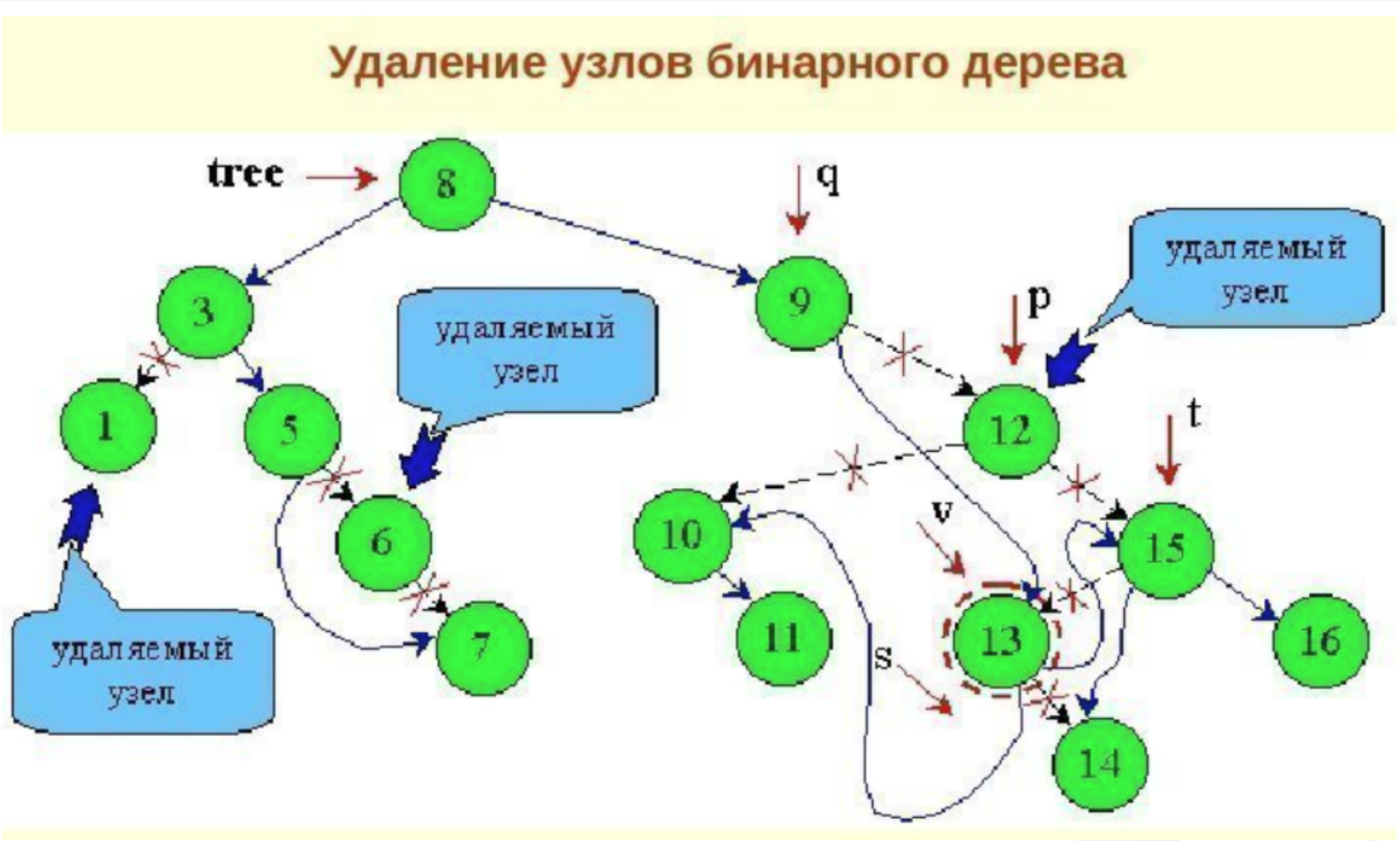
г)

Удаление узла из двоичного дерева поиска

Процесс удаления можно разбить на этапы:

- найти узел, который будет поставлен на место удаляемого;
- реорганизовать дерево так, чтобы не нарушились его свойства;
- присоединить новый узел к узлу-предку удаляемого узла;
- освободить память из-под удаляемого узла.

Удаление узла из двоичного дерева поиска



Удаление узла из двоичного дерева поиска

```
enum Dir    { LEFT, RIGHT };

Node* descent( Node* p ) {          // ---------------------------- Спуск по дереву
    Node* prev, *y = p->right;
    if ( !y->left ) y->left = p->left;          // 1
    else {
        do { prev = y; y = y->left; }           // 2
        while( y->left );
        y->left    = p->left;                   // 3
        prev->left  = y->right;                 // 4
        y->right    = p->right;                 // 5
    }
    return y;
}

// ------------------------------------------------------------ Удаление узла дерева
Node* remove_node( Node* root, Node* p, Node* parent, Dir dir ) {
    Node *y;                    // Узел, заменяющий удаляемый
    if  ( !p->left )        y = p->right;               // 11
    else if ( !p->right )   y = p->left;                // 12
    else                    y = descent( p );           // 13
    if ( p == root ) root = y;                          // 14
    else {                                              // 15
        if ( dir == LEFT )  parent->left = y;
        else                parent->right = y;
    }
    delete p;                                           // 16
    return root;
}
```

Удаление узла из двоичного дерева поиска

Рассмотрим реализацию этого алгоритма в программе remove_node. В функцию передается указатель на удаляемый узел p. Сначала находим указатель на узел y, который должен заменить удаляемый. Если у узла p нет левого поддерева, на его место будет поставлена вершина (возможно, пустая) его правого поддерева (*оператор 11*). Иначе, если у узла p нет правого поддерева, на его место будет поставлена вершина его левого поддерева (*оператор 12*).

В противном случае оба поддерева узла существуют, и для определения заменяющего узла вызывается вспомогательная функция descent, выполняющая спуск по дереву. В этой функции прежде всего проверяется особый случай, описанный выше (*оператор 1*). Если же условие отсутствия левого потомка у правого потомка удаляемого узла не выполняется, организуется цикл (*оператор 2*), на каждой итерации которого указатель на текущий узел запоминается в переменной prev, а указатель y смещается вниз и влево по дереву до того момента, пока не станет ссылаться на узел, не имеющий левого потомка — он-то нам и нужен!

Удаление узла из двоичного дерева поиска

В *операторе 3* к этой пустующей ссылке присоединяется левое поддерево удаляемого узла. Перед тем, как присоединять к этому узлу правое поддерево удаляемого узла (*оператор 5*), требуется «пристроить» его собственное правое поддерево. Мы присоединяем его к левому поддереву предка заменяющего узла *y* (*оператор 4*), поскольку этот узел перейдет на новое место.

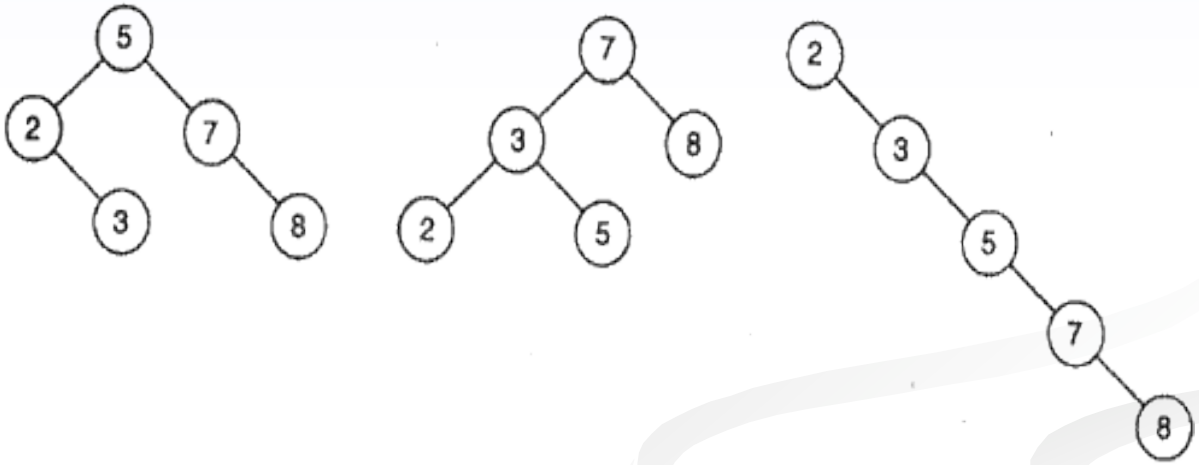
Функция descent возвращает указатель на узел, заменяющий удаляемый. Если мы удаляем корень, надо обновить указатель на него (*оператор 14*), иначе — присоединить этот узел к соответствующему поддереву предка удаляемого узла (*оператор 15*). После удаления узла освобождаем память (*оператор 16*).

Двоичное дерево поиска

- **Балансировка дерева**
- Если пути в дереве от корня до листьев имеют примерно одинаковую длину, т.е. чтобы глубина левого и правого поддеревьев была примерно одинакова для любого узла. В противном случае теряется производительность.

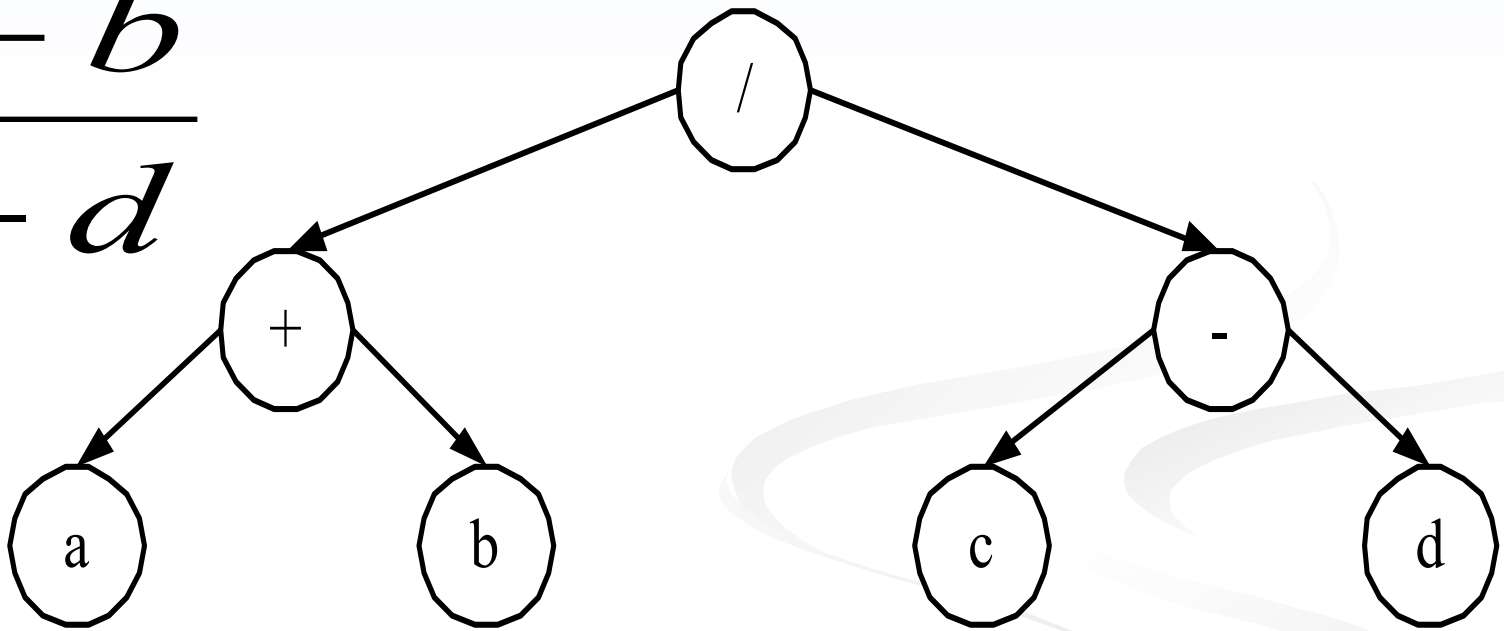
- В вырожденном случае может оказаться, что все левое дерево пусто на каждом уровне, есть только правые деревья, и в таком случае дерево вырождается в односвязный список.

Двоичное дерево поиска



Дерево арифметического выражения

$$\frac{a+b}{c-d}$$



Способы обхода дерева

1. Прямой порядок прохождения (префиксная запись `/+ a b - c d)`:

- попасть в корень;
- пройти левое поддерево;
- пройти правое поддерево.

Способы обхода дерева

2. Симметричный порядок прохождения (инфиксная запись `(a+b)/(c-d)`):

- пройти левое поддерево;
- попасть в корень;
- пройти правое поддерево.

Способы обхода дерева

3. Обратный порядок прохождения (постфиксная запись **a b+ c d - /**):

- пройти левое поддерево;
- пройти правое поддерево;
- попасть в корень.

Дополнительно

.1.На нашем сайте визуализатор построения дерева

- На сайте Питсбургского университета – слайды к лекциям и примеры программ
- ***Стэнфордский*** университет

http://www.stanford.edu/~blp/avl/libavl.html/Binary-Search-Trees.html

- С-П университет информатики, точ. мех. и оптики

http://rain.ifmo.ru/cat/view.php/vis/trees/balanced-bst-2008